บทที่ 6

การจัดลำดับการใช้หน่วยประมวลผลกลาง

ในการทำงานของระบบคอมพิวเตอร์ โปรแกรมหรือแอปพลิเคชันที่ผู้ใช้สั่งให้ทำงานจะถูกแปลงเป็น*โพรเซส (process)* ซึ่งแต่ละโพรเซสจำเป็นต้องอาศัยทรัพยากรของระบบ (system resources) เช่น หน่วยประมวลผลกลาง (CPU) หน่วยความจำ (memory) อุปกรณ์นำเข้า-ส่งออก (I/O devices) และไฟล์ต่าง ๆ เพื่อให้สามารถทำงานได้สำเร็จ อย่างไรก็ตาม ทรัพยากรเหล่านี้มีจำนวนจำกัด โดยไม่สามารถรองรับทุกโพรเซสได้พร้อมกันทั้งหมด ระบบปฏิบัติการจึงมีบทบาทสำคัญในการจัดสรรทรัพยากรอย่างเป็นธรรมและมีประสิทธิภาพ เพื่อให้โพรเซสต่างๆ สามารถดำเนินงานร่วมกันได้โดยไม่เกิดความขัดแย้ง

หนึ่งในทรัพยากรที่มีความสำคัญสูงสุดคือหน่วยประมวลผลกลาง ซึ่งเป็นหัวใจหลักของการทำงานทั้งหมด หากไม่มีการจัดการอย่างเหมาะสม จะทำให้บางโพรเซสได้รับการประมวลผลล่าช้า หรืออาจเกิดการ*อดตาย (starvation)* คือไม่ได้รับโอกาสใช้งานหน่วยประมวลผลกลางเลย เพื่อแก้ปัญหานี้ ระบบปฏิบัติการใช้กลไกที่เรียกว่า**การจัดลำดับการใช้หน่วยประมวลผลกลาง (CPU scheduling)**

เมื่อโพรเซสถูกสร้างขึ้นและพร้อมที่จะทำงาน แต่ยังไม่ได้รับการทำงานร่วมกับหน่วยประมวลผลกลาง โพรเซสนั้นจะถูกจัดเก็บไว้ใน*คิวพร้อม (ready queue)* ซึ่งทำหน้าที่เหมือนห้องรอคอยสำหรับโพรเซสที่ต้องการใช้หน่วยประมวลผลกลาง แต่ยังไม่ถึงคิวเข้าใช้งาน เมื่อหน่วยประมวลผลกลางว่าง ตัวกำหนดการใช้งานหน่วยประมวลผลกลาง (CPU scheduler) จะเลือกโพรเซสจากคิวพร้อมขึ้นมาหนึ่งตัวตาม*อัลกอริทึมการจัดลำดับ (scheduling algorithm)* ที่กำหนดไว้ แล้วมอบสิทธิ์การใช้งานหน่วยประมวลผลกลางให้กับโพรเซสนั้น โพรเซสจะเปลี่ยนสถานะจาก *พร้อม (ready)* เป็น *กำลังทำงาน (running)* และเริ่มต้นการประมวลผลจริง

6.1 หลักความต้องการพื้นฐานของการจัดลำดับ

จุดประสงค์หลักของการใช้*ระบบหลายโปรแกรม (multiprogramming)* คือ การทำให้หน่วยประมวลผลกลางทำงานอย่างต่อเนื่องโดยไม่ปล่อยให้ว่าง และเพิ่มประสิทธิภาพการใช้งานให้สูงที่สุด เนื่องจากหน่วยประมวลผลกลางถือเป็นทรัพยากรที่สำคัญและมีค่ามากที่สุดในระบบคอมพิวเตอร์

ในระบบที่มีหน่วยประมวลผลกลางเพียงตัวเดียว การทำงานในช่วงเวลาใดเวลาหนึ่งจะถูกครอบครองโดยโพรเซสเพียงตัวเดียวเท่านั้น หากในระบบมีหลายโปรแกรมหรืองานที่ต้องรันพร้อมกัน โปรแกรมที่เหลือจำเป็นต้องรอการจัดสรรเวลาเพื่อเข้าใช้งานหน่วยประมวลผลกลาง หลักการนี้ไม่ซับซ้อนมากนัก โดยทั่วไปโพรเซสจะดำเนินการไปเรื่อย ๆ จนกว่าจะถึงจุดที่ต้องรอการทำงานอื่น ๆ เช่น การร้องขออินพุต/เอาต์พุต (I/O request) เป็นต้น

หากระบบสามารถรันได้เพียงโปรแกรมเดียว เมื่อโพรเซสต้องหยุดรอ I/O หน่วยประมวลผลกลางก็จะหยุดทำงานไปด้วย ซึ่งเท่ากับว่าเกิด*การเสียเวลาของระบบ (idle time)* โดยที่ทรัพยากรสำคัญอย่างหน่วย

ประมวลผลกลางไม่ได้ถูกใช้งาน ดังนั้น หลักการของการรันหลายโปรแกรมจึงถูกออกแบบมาเพื่อแก้ปัญหานี้ กล่าวคือ เมื่อโปรแกรมหนึ่งหยุดรอการทำงาน I/O ระบบจะจัดสรรหน่วยประมวลผลกลางให้กับโปรแกรมอื่นที่อยู่ในคิวพร้อมเข้ามาทำงานแทนทันที

ด้วยวิธีนี้หน่วยประมวลผลกลางจะไม่ถูกปล่อยให้ว่าง แต่จะมีงานให้ทำอย่างต่อเนื่องสลับไปเรื่อย ๆ ระหว่างโปรแกรมหลายตัวในระบบ ซึ่งถือเป็นพื้นฐานของแนวคิดการจัดลำดับ (scheduling) และเป็นหัวใจของระบบปฏิบัติการสมัยใหม่ที่สามารถรองรับการทำงานหลายโปรแกรมพร้อมกันได้

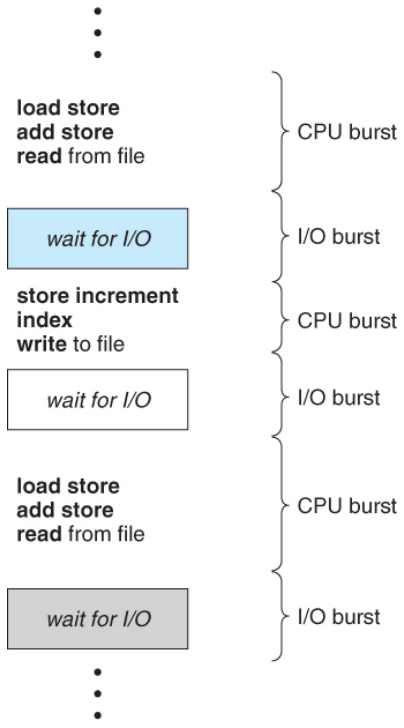
6.1.1 ช่วงเวลาของอินพุต/เอาต์พุตและการใช้หน่วยประมวลผลกลาง

ความสำคัญของการจัดเวลาหน่วยประมวลผลกลาง เกิดจากธรรมชาติของการทำงานของโพรเซส ซึ่งมักประกอบด้วย 2 ช่วงเวลาหลัก ดังนี้

1. **CPU Burst** – ช่วงเวลาที่โพรเซสใช้หน่วยประมวลผลกลางเพื่อประมวลผลคำสั่ง เช่น การคำนวณเชิงคณิตศาสตร์ การประมวลผลข้อมูล หรือการทำงานทางตรรกะ

2. **I/O Burst** – ช่วงเวลาที่โพรเซสหยุดรออุปกรณ์ภายนอก เช่น ดิสก์ คีย์บอร์ด หรือจอแสดงผล เพื่อรับหรือส่งข้อมูล

โพรเซสจะทำงานโดยสลับไปมาระหว่างสองช่วงนี้ ซึ่งไม่ได้เกิดขึ้นพร้อมกัน โดยทั่วไปมักเริ่มต้นด้วย CPU Burst ตามด้วยการรอ I/O เมื่อ I/O เสร็จสิ้น โพรเซสก็จะกลับมาเข้าสู่ CPU Burst อีกครั้ง สลับกันไปเรื่อย ๆ จนกว่างานจะเสร็จสมบูรณ์ ดังแสดงในรูปที่ 6.1



รูปที่ 6.1: การทำงานที่สลับระหว่างช่วงการใช้หน่วยประมวลผลกลางและช่วงรอ I/O

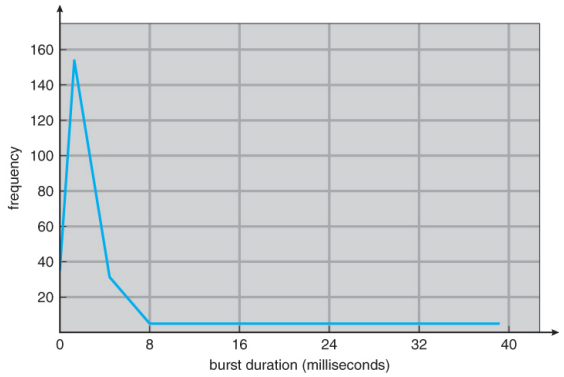
จากรูป นิสิตจะเห็นได้ว่า การทำงานของโพรเซสไม่ได้ใช้หน่วยประมวลผลกลางอย่างต่อเนื่องตลอดเวลา แต่จะมีการหยุดพักเพื่อรอ I/O อยู่เสมอ นี่คือสาเหตุที่ระบบจำเป็นต้องจัดสรรหน่วยประมวลผลกลางให้กับโพรเซสอื่นในระหว่างที่บางโพรเซสกำลังรอ

6.1.2 ลักษณะการกระจายของเวลาในการใช้หน่วยประมวลผลกลาง

จากการศึกษาลักษณะการทำงานของโพรเซส พบว่าการใช้หน่วยประมวลผลกลางมักมีรูปแบบดังนี้

- ส่วนใหญ่โพรเซสจะใช้เวลาหน่วยประมวลผลกลางเป็น*ช่วงสั้น ๆ (short CPU Burst)* แต่เกิดขึ้นบ่อย
- การใช้หน่วยประมวลผลกลางที่เป็นช่วงยาว (long CPU Burst) จะเกิดขึ้นไม่บ่อยนัก
- โพรเซสที่มีการติดต่อกับอุปกรณ์ I/O บ่อย ๆ เช่น โปรแกรมเชิงโต้ตอบ (interactive program) มักมี CPU Burst สั้น ๆ แต่ทำซ้ำถี่มาก
- โพรเซสที่เน้นการคำนวณเชิงคณิตศาสตร์หรือการประมวลผลข้อมูล (CPU-bound process) จะมี CPU Burst ยาวและใช้งานหน่วยประมวลผลกลางมากกว่าโพรเซสอื่น

ลักษณะนี้สามารถสรุปได้จากกราฟในรูปที่ 6.2 ซึ่งแสดงความถี่ของการใช้หน่วยประมวลผลกลางกับช่วงเวลาที่ใช้



รูปที่ 6.2: การกระจายความถี่ของการใช้เวลาหน่วยประมวลผลกลางของโพรเซส

จากกราฟจะเห็นว่า *CPU Burst สั้นเกิดขึ้นบ่อย* ในขณะที่ *CPU Burst ยาวเกิดขึ้นไม่บ่อย* การทำความเข้าใจลักษณะนี้มีความสำคัญต่อการออกแบบอัลกอริทึมการจัดเวลาหน่วยประมวลผลกลาง เพราะช่วยให้เราสามารถเลือกใช้อัลกอริทึมที่เหมาะสมกับลักษณะงานในระบบได้ เช่น

- งานที่เน้น **I/O-bound** ควรใช้วิธีที่ช่วยให้โพรเซสได้หน่วยประมวลผลกลางอย่างรวดเร็วเพื่อตอบสนองผู้ใช้
- งานที่เน้น **CPU-bound** อาจต้องการอัลกอริทึมที่ช่วยให้โพรเซสใหญ่ ๆ ได้รับเวลาหน่วยประมวลผลกลางอย่างยุติธรรมโดยไม่กีดกันงานอื่น

แนวคิดของการจัดลำดับการใช้หน่วยประมวลผลกลาง ไม่ได้มีเพียงการแบ่งเวลาหน่วยประมวลผลกลางเท่านั้น แต่ยังต้องคำนึงถึง*ธรรมชาติของโพรเซสแต่ละประเภท* เพื่อสร้างสมดุลระหว่าง *ประสิทธิภาพ (efficiency) ความยุติธรรม (fairness) และความตอบสนอง (responsiveness)* ของระบบ

6.2 ตัวจัดเวลาหน่วยประมวลผลกลาง

เมื่อใดก็ตามที่หน่วยประมวลผลกลางว่างลง ระบบปฏิบัติการจะต้องตัดสินใจเลือกโพรเซสหนึ่งจากคิวพร้อมเพื่อเข้ามาใช้งานหน่วยประมวลผลกลาง การเลือกโพรเซสนี้ถูกจัดการโดย *ตัวจัดการช่วงสั้น (Short-term scheduler)* หรือที่นิยมเรียกว่า *ตัวจัดเวลาหน่วยประมวลผลกลาง (CPU scheduler)*

หน้าที่ของ CPU scheduler คือการเลือกโพรเซสที่อยู่ในหน่วยความจำหลักและพร้อมที่สุดสำหรับการประมวลผล แล้วมอบสิทธิ์การใช้งานหน่วยประมวลผลกลาง และทรัพยากรที่เกี่ยวข้องให้ โพรเซสที่ถูกเลือกจะเปลี่ยนสถานะจาก *ready* ไปเป็น *running* และเริ่มดำเนินงาน

การเลือกโพรเซสนี้ไม่ได้จำกัดว่าต้องเป็นแบบมาก่อน-ทำก่อน (First In First Out: FIFO) เสมอไป แต่สามารถใช้หลักเกณฑ์อื่นได้ เช่น การจัดตามความสำคัญ (priority scheduling) การใช้โครงสร้างต้นไม้ (Tree) หรือการใช้โครงสร้างลิงก์ลิสต์ (Linked List) เพื่อจัดการโพรเซสในคิว สิ่งสำคัญคือโพรเซสทุกตัวที่พร้อมใช้งานหน่วยประมวลผลกลาง จะต้องได้รับโอกาสในการเข้าถึงหน่วยประมวลผลกลางภายในระยะเวลาที่เหมาะสม เพื่อให้เกิดความเป็นธรรมและเพิ่มประสิทธิภาพของระบบโดยรวม

เมื่อโพรเซสหนึ่งถูกนำออกจากหน่วยประมวลผลกลาง ระบบต้องเก็บข้อมูลการทำงานของโพรเซสนั้นไว้ เช่น ค่าของรีจิสเตอร์ สถานะการทำงาน หรือคำสั่งที่กำลังประมวลผลอยู่ ข้อมูลเหล่านี้ถูกบันทึกใน*บล็อกควบคุมโพรเซส* (Process Control Block: PCB) เพื่อให้สามารถเรียกคืน (resume) โพรเซสได้จากจุดเดิมโดยไม่สูญเสียความต่อเนื่องของการทำงาน

การให้สิทธิ์การจัดเวลา (Preemptive Scheduling)

การให้สิทธิ์การจัดเวลาเป็นกระบวนการที่ระบบปฏิบัติการตัดสินใจว่าโพรเซสใดควรได้รับหน่วยประมวลผลกลาง ซึ่งขึ้นอยู่กับสถานการณ์ที่เกิดขึ้นระหว่างการทำงาน โดยหลัก ๆ แบ่งได้เป็น 4 กรณี ดังนี้

1. เมื่อโพรเซสเปลี่ยนสถานะจาก *running* ไปเป็น *waiting* เช่น ในกรณีที่ต้องรอ I/O หรือรอให้โพรเซสลูก (child process) เสร็จสิ้นก่อน
2. เมื่อโพรเซสเปลี่ยนสถานะจาก *running* ไปเป็น *ready* เช่น เมื่อเกิด *Interrupt* ที่ทำให้โพรเซสถูกแย่งหน่วยประมวลผลกลาง
3. เมื่อโพรเซสเปลี่ยนสถานะจาก *waiting* ไปเป็น *ready* เช่น เมื่อการทำงาน I/O เสร็จสิ้นและโพรเซสกลับมาพร้อมใช้งาน
4. เมื่อโพรเซสสิ้นสุดการทำงาน (termination) ระบบต้องเลือกโพรเซสใหม่มาแทนที่

ในทั้ง 4 สถานการณ์ที่กล่าวมา สถานการณ์ที่ 1 และ 4 เป็น *สถานการณ์บังคับ* คือระบบต้องเลือกโพรเซสใหม่มาใช้หน่วยประมวลผลกลาง ทันทีเนื่องจากโพรเซสเดิมไม่สามารถทำงานต่อได้แล้ว ส่วนสถานการณ์ที่ 2 และ 3 ขึ้นอยู่กับนโยบายและอัลกอริทึมของการจัดเวลาในระบบ จากมุมมองของวิธีการจัดเวลา สามารถแบ่งได้เป็น 2 แบบใหญ่ ๆ ได้แก่

- **Non-preemptive scheduling** – โพรเซสที่ได้ครอบครองหน่วยประมวลผลกลาง จะทำงานต่อไปจนกว่าจะเสร็จสิ้น หรือจนกว่าจะเปลี่ยนสถานะเป็น *waiting* โดยไม่มีการแย่งหน่วยประมวลผลกลาง วิธีนี้มักใช้ในระบบที่ต้องการความเรียบง่าย เช่น ระบบแบบ batch processing หรือฮาร์ดแวร์บางประเภท

- **Preemptive scheduling** – ระบบสามารถแย่งหน่วยประมวลผลกลาง จากโพรเซสหนึ่งเพื่อมอบให้โพรเซสอื่นได้ เช่น เมื่อมีโพรเซสที่มีความสำคัญสูงกว่ามาใหม่ วิธีนี้เหมาะสำหรับระบบที่ต้องการการตอบสนองที่รวดเร็ว (responsiveness) เช่น ระบบแบบ multi-user หรือระบบ interactive

ตัวส่งต่อ (Dispatcher)

องค์ประกอบสำคัญอีกตัวที่เกี่ยวข้องกับการจัดเวลาคือ **ตัวส่งต่อ (dispatcher)** ซึ่งเป็นโมดูลที่ทำหน้าที่ควบคุมการโอนสิทธิ์การใช้งานหน่วยประมวลผลกลาง จากโพรเซสหนึ่งไปยังอีกโพรเซสหนึ่ง โดยหน้าที่หลักของ Dispatcher ได้แก่

1. การย้าย Context Switching – เก็บค่าของโพรเซสเดิมและเรียกคืนค่าของโพรเซสใหม่
2. การย้ายไปยัง User Mode – เพื่อให้โพรเซสที่ทำงานเป็นโปรแกรมผู้ใช้สามารถรันได้อย่างปลอดภัย
3. การกระโดดไปยังตำแหน่งคำสั่งที่ถูกต้องในโปรแกรม เพื่อให้โพรเซสดำเนินการต่อจากจุดที่หยุดไว้

ทุกครั้งที่มีการเปลี่ยนโพรเซส Dispatcher จะต้องทำงาน กระบวนการนี้เรียกว่า Context Switch ซึ่งเป็นขั้นตอนที่มีค่าใช้จ่าย (Overhead) เพราะ หน่วยประมวลผลกลางต้องเสียเวลาในการบันทึกและกู้คืนสถานะก่อนที่จะกลับไปทำงานต่อ ดังนั้น ความเร็วของ Dispatcher จึงมีความสำคัญมาก เวลาที่ใช้ไปในการเปลี่ยนโพรเซสนี้เรียกว่า **Dispatch Latency** ซึ่งหมายถึงช่วงเวลาที่ หน่วยประมวลผลกลาง ไม่สามารถทำงานให้โพรเซสใด ๆ ได้เพราะกำลังเปลี่ยนงานอยู่

กล่าวโดยสรุป CPU Scheduler ทำหน้าที่เป็น “ผู้ตัดสินใจ” ในการเลือกโพรเซส ส่วน Dispatcher ทำหน้าที่เป็น “ผู้ปฏิบัติ” ที่ทำให้การเปลี่ยนงานเกิดขึ้นจริง การทำงานที่ผสมผสานกันของทั้งสององค์ประกอบนี้ทำให้ระบบปฏิบัติการสามารถบริหารจัดการหน่วยประมวลผลกลางได้อย่างมีประสิทธิภาพและตอบสนองความต้องการของผู้ใช้ในสถานการณ์ต่าง ๆ

6.3 เกณฑ์การวิเคราะห์ประสิทธิภาพ

อัลกอริทึมที่ใช้ในการเลือกโพรเซสเพื่อเข้ามาใช้งานหน่วยประมวลผลกลาง (CPU scheduling algorithms) มีหลากหลายรูปแบบ โดยแต่ละอัลกอริทึมจะมีคุณสมบัติ จุดเด่น และข้อจำกัดแตกต่างกันไป ดังนั้น การเลือกใช้อัลกอริทึมที่เหมาะสมกับระบบหรือสภาพแวดล้อมการทำงานใด ๆ จำเป็นต้องพิจารณาเป้าหมายและเกณฑ์ด้านประสิทธิภาพอย่างรอบคอบ

เกณฑ์เหล่านี้ถือเป็นปัจจัยชี้วัดสำคัญว่าอัลกอริทึมการจัดเวลาที่ใช้นั้นมีประสิทธิภาพเพียงใด และเหมาะสมกับลักษณะงานที่ต้องประมวลผลหรือไม่ โดยทั่วไป เกณฑ์ที่นิยมใช้วิเคราะห์ ได้แก่

1. **การใช้งานหน่วยประมวลผลกลาง (CPU utilization)** หน่วยประมวลผลกลางควรถูกใช้งานให้มากที่สุดเท่าที่เป็นไปได้ เพื่อหลีกเลี่ยงการปล่อยให้หน่วยประมวลผลกลางว่าง (idle) โดยเปล่าประโยชน์ โดยทั่วไปการใช้งาน หน่วยประมวลผลกลาง ควรอยู่ในช่วงประมาณ 40% ถึง 90% ขึ้นอยู่กับปริมาณงานที่เข้าสู่ระบบ หากการใช้งานต่ำเกินไป หมายถึงมีทรัพยากรถูกปล่อยทิ้งไว้โดยไม่ได้ใช้งาน แต่หากสูงจนเกินไป (เกือบ 100%) อาจสะท้อนว่าระบบถูกใช้งานหนักมากและอาจเกิดคอขวด (bottleneck) ได้
2. **ปริมาณงานสูงสุด (Throughput)** หมายถึงจำนวนโพรเซสที่สามารถประมวลผลเสร็จสิ้นได้ต่อหนึ่งหน่วยเวลา ค่า Throughput ที่สูงสะท้อนถึงความสามารถของระบบในการรองรับงานจำนวนมาก

เกณฑ์นี้มีความสำคัญอย่างยิ่งในระบบที่เน้นการประมวลผลงานจำนวนมาก เช่น ระบบแบบ batch processing หรือศูนย์ประมวลผลข้อมูล (data center)

3. **เวลาครบวงจรน้อยที่สุด (Turnaround Time)** คือเวลาทั้งหมดที่โพรเซสใช้ตั้งแต่เข้าสู่ระบบจนกระทั่งเสร็จสิ้นการทำงานและได้ผลลัพธ์กลับมา โดยคิดรวมทั้งเวลาประมวลผล (CPU Time) เวลารอในคิว (Waiting Time) และเวลาที่เกี่ยวข้องกับ I/O ด้วย ผู้ใช้มักคาดหวังว่าโพรเซสที่สั่งให้ทำงานจะเสร็จโดยเร็วที่สุด ดังนั้น เป้าหมายของระบบปฏิบัติการคือการลดค่า Turnaround Time ให้น้อยที่สุด

4. **เวลารอน้อยที่สุด (Waiting Time)** หมายถึงเวลาที่โพรเซสใช้ในการรอคอยอยู่ในคิวพร้อม โดยไม่ได้รับการประมวลผลจากหน่วยประมวลผลกลาง หากเวลารอนานเกินไปจะทำให้ระบบตอบสนองได้ช้าและอาจทำให้เกิดความไม่เป็นธรรมระหว่างโพรเซส การลด waiting time จึงช่วยให้โพรเซสแต่ละตัวได้รับการบริการที่รวดเร็วขึ้นและระบบทำงานสมดุลมากขึ้น

5. **เวลาตอบสนองน้อยที่สุด (Response Time)** คือเวลาที่ระบบใช้ในการตอบสนองครั้งแรกหลังจากผู้ใช้ร้องขอหรือป้อนคำสั่งเข้าสู่ระบบ โดยไม่จำเป็นต้องรอจนโพรเซสเสร็จสิ้นทั้งหมด ตัวอย่างเช่น เมื่อผู้ใช้พิมพ์คำสั่งในระบบปฏิบัติการแบบโต้ตอบ (interactive system) หรือคลิกเมาส์ในโปรแกรมกราฟิก ผู้ใช้คาดหวังว่าจะได้รับการตอบสนองกลับมาในทันที ดังนั้น การลด Response Time จึงเป็นหัวใจสำคัญของระบบที่เน้นการโต้ตอบกับผู้ใช้

การวิเคราะห์ประสิทธิภาพของอัลกอริทึมการจัดเวลาหน่วยประมวลผลกลาง จำเป็นต้องพิจารณาเกณฑ์เหล่านี้อย่างรอบด้าน ไม่ใช่มุ่งเน้นเพียงตัวใดตัวหนึ่ง เนื่องจากการปรับปรุงในด้านหนึ่งอาจทำให้ด้อยประสิทธิภาพในอีกด้านหนึ่ง เช่น การลด Response Time อาจเพิ่มค่า Waiting Time ของโพรเซสอื่น ๆ ดังนั้น ระบบปฏิบัติการต้องสร้างสมดุลที่เหมาะสมตามลักษณะงานและวัตถุประสงค์ของระบบที่ออกแบบ

6.4 อัลกอริทึมการจัดลำดับเวลา

อัลกอริทึมที่ใช้ในการจัดสรรเวลาของหน่วยประมวลผลกลาง มีบทบาทสำคัญในการตัดสินใจว่าโพรเซสใดควรได้รับสิทธิ์การใช้งานหน่วยประมวลผลกลางก่อน โพรเซสเหล่านี้จะถูกจัดเก็บอยู่ในคิวพร้อม และรอให้ระบบปฏิบัติการจัดลำดับให้เข้าถึงหน่วยประมวลผลกลาง

การเลือกใช้อัลกอริทึมขึ้นอยู่กับเป้าหมายของระบบ เช่น ต้องการความเป็นธรรม (fairness) ความรวดเร็วในการตอบสนอง (responsiveness) หรือการเพิ่มปริมาณงานที่เสร็จสิ้น (throughput) เป็นต้น ในหัวข้อนี้เราจะเริ่มจากการศึกษาวิธีการจัดเวลาที่ง่ายที่สุดและเป็นพื้นฐานที่สุดก่อน คืออัลกอริทึมแบบมาก่อนได้ก่อน

6.4.1 อัลกอริทึมแบบมาก่อนได้ก่อน (First-Come, First-Served (FCFS))

อัลกอริทึมแบบมาก่อนได้ก่อน (First-Come, First-Served: FCFS) ถือเป็นรูปแบบการจัดลำดับที่ง่ายและตรงไปตรงมาที่สุด โดยมีหลักการว่า *โพรเซสใดที่ร้องขอการใช้งานหน่วยประมวลผลกลางก่อน จะได้รับสิทธิ์ในการใช้งานก่อนเสมอ* กลไกการทำงานของ FCFS คือ

- เมื่อโพรเซสเข้าสู่ระบบ จะถูกจัดเก็บไว้ที่ท้ายคิวพร้อม (ready queue)
- เมื่อหน่วยประมวลผลกลางว่าง ระบบจะมอบสิทธิ์ให้กับโพรเซสที่อยู่ต้นคิวทันที

- โพรเซสจะทำงานไปจนกว่าจะเสร็จสิ้น หรือจนกว่าจะเปลี่ยนสถานะเป็นคอย (waiting) โดยไม่สามารถถูกแทรกกลางได้

ดังนั้น FCFS เป็นอัลกอริทึมที่มีลักษณะเป็น Non-preemptive กล่าวคือ ไม่มีการขัดจังหวะการทำงานของโพรเซส

ตัวอย่างที่ 1 : สมมติว่าระบบคอมพิวเตอร์มี 3 โพรเซส ได้แก่ P_1, P_2, และ P_3 ซึ่งต้องการเข้าใช้งานหน่วยประมวลผลกลาง โดยมีข้อมูลดังนี้:

1. โพรเซส P_1 เข้าสู่ระบบเวลา 8.00 และต้องการใช้หน่วยประมวลผลกลาง จำนวน 2 หน่วยเวลา
2. โพรเซส P_2 เข้าสู่ระบบเวลา 8.10 และต้องการใช้หน่วยประมวลผลกลาง จำนวน 1 หน่วยเวลา
3. โพรเซส P_3 เข้าสู่ระบบเวลา 8.25 และต้องการใช้หน่วยประมวลผลกลาง จำนวน 0.25 หน่วยเวลา

ผลการทำงานสามารถสรุปได้ดังตารางต่อไปนี้:

Process	Arrival Time	Run Time	Start Time	Finish Time	Turnaround Time
P_1	8.00	2.00	8.00	10.00	2.00
P_2	8.10	1.00	10.00	11.00	2.90
P_3	8.25	0.25	11.00	11.25	3.00
					7.90

ค่าเฉลี่ยของ Turnaround Time = 7.90/3 = 2.63 หน่วยเวลา

ตัวอย่างที่ 2 : เมื่อพิจารณาระบบที่มี 3 โพรเซสคือ P_1, P_2, และ P_3 ซึ่งเข้าสู่ระบบตามลำดับ และมี Burst Time ดังนี้:

Process	Burst Time
P_1	24
P_2	3
P_3	3

การจัดลำดับแบบ FCFS จะได้ Gantt Chart ดังนี้:



- P_1: รอ = 0 หน่วยเวลา
- P_2: รอ = 24 หน่วยเวลา
- P_3: รอ = 27 หน่วยเวลา

ดังนั้น ค่าเฉลี่ยเวลารอ = (0 + 24 + 27)/3 = 17 หน่วยเวลา แต่ถ้าสลับลำดับเป็น P_2, P_3, P_1 จะได้ Gantt Chart ดังนี้:



เวลารอแต่ละโพรเซสคือ $P_2 = 0$, $P_3 = 3$, และ $P_1 = 6$ ดังนั้น ค่าเฉลี่ยเวลารอ = (0 + 3 + 6)/3 = 3 หน่วยเวลา

การวิเคราะห์อัลกอริทึม FCFS

ข้อดีของ FCFS คือ ความง่ายและความเป็นธรรมในเชิงลำดับเวลา เพราะโพรเซสที่เข้ามาก่อนย่อมได้รับสิทธิ์ก่อน อย่างไรก็ตาม FCFS มีข้อเสียที่สำคัญคือปัญหา *convoy effect* ซึ่งเกิดขึ้นเมื่อโพรเซสขนาดใหญ่ที่ต้องใช้หน่วยประมวลผลกลางนานมากเข้ามาก่อน ทำให้โพรเซสขนาดเล็กที่ตามมาถูกบังคับให้รอเป็นเวลานาน

ด้วยเหตุนี้ FCFS จึงไม่เหมาะกับระบบที่ต้องการการตอบสนองอย่างรวดเร็ว เช่น ระบบ interactive หรือระบบ multi-user แต่ยังคงเหมาะสมกับระบบแบบ batch processing ที่เน้นความเรียบง่ายและไม่ต้องการการโต้ตอบทันที

6.4.2 Shortest-Job-First (SJF)

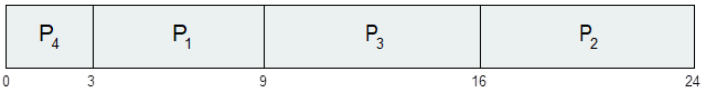
จากการใช้อัลกอริทึมมาก่อนได้รับบริการก่อน (FCFS) จะเห็นว่าค่าเฉลี่ยของ*เวลาครบวงจร (Turnaround Time)* และ *เวลารอ (Waiting Time)* มักมีค่าสูง โดยเฉพาะอย่างยิ่งเมื่อในคิวพร้อม (ready queue) มีโพรเซสที่ใช้เวลาในการประมวลผลแตกต่างกันมาก โพรเซสที่ใช้หน่วยประมวลผลกลางนาน (long job) มักทำให้โพรเซสขนาดเล็ก (short job) ต้องรอเป็นเวลานาน และเกิดปัญหาที่เรียกว่า *convoy effect*

เพื่อแก้ไขปัญหานี้ มีการใช้อัลกอริทึม **Shortest-Job-First (SJF)** ซึ่งมีหลักการคือ *ให้โพรเซสที่ใช้ CPU Burst สั้นที่สุด ได้รับสิทธิ์ใช้งานก่อน* วิธีนี้สามารถช่วยลดค่าเฉลี่ยของ turnaround time และ waiting time ได้อย่างมีนัยสำคัญ

ตัวอย่างที่ 3 : สมมติว่าระบบมีโพรเซส P_1, P_2, P_3 และ P_4 เข้ามาพร้อมกัน โดยมี Burst Time ดังนี้:

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

หากใช้ SJF จะได้ลำดับการทำงานตาม Gantt Chart:



เวลารอของแต่ละโพรเซสคือ:

- P_1: รอ 3 หน่วยเวลา
- P_2: รอ 16 หน่วยเวลา
- P_3: รอ 9 หน่วยเวลา
- P_4: รอ 0 หน่วยเวลา

ดังนั้น ค่าเฉลี่ยเวลารอ = (3+16+9+0)/4 = 7 หน่วยเวลา

จากตัวอย่างที่ 1 หากใช้ SJF คำนวณค่าเฉลี่ยของเวลาครบวงจร (Turnaround Time) สามารถทำได้ดังตาราง:

Process	Arrival Time	Run Time	Start Time	Finish Time	Turnaround Time
P_1	8.00	2.00	8.00	10.00	2.00
P_2	8.10	1.00	10.25	11.25	3.15
P_3	8.25	0.25	10.00	10.25	2.00
					7.15

ค่าเฉลี่ย Turnaround Time = 7.15/3 = 2.38 หน่วยเวลา

SJF แบบ Preemptive และ Non-Preemptive

- *Non-Preemptive SJF*: เมื่อโพรเซสได้รับหน่วยประมวลผลกลางแล้ว จะทำงานจนเสร็จสิ้น แม้จะมีโพรเซสใหม่เข้ามาซึ่งใช้เวลาสั้นกว่าก็ตาม
- *Preemptive SJF* หรือที่มักเรียกว่า *Shortest-Remaining-Time-First (SRTF)*: หากมีโพรเซสใหม่เข้ามาที่มี CPU Burst สั้นกว่าของโพรเซสที่กำลังทำงานอยู่ ระบบจะหยุด (Preempt) โพรเซสเดิมและสลับไปให้โพรเซสใหม่ทำงานก่อน

ตัวอย่างที่ 4 : สมมติว่าระบบมีโพรเซส 3 ตัวดังนี้:

- P_1: เข้าระบบเวลา 0.0 ต้องใช้หน่วยประมวลผลกลาง 8 หน่วยเวลา
- P_2: เข้าระบบเวลา 0.4 ต้องใช้หน่วยประมวลผลกลาง 4 หน่วยเวลา
- P_3: เข้าระบบเวลา 1.0 ต้องใช้หน่วยประมวลผลกลาง 1 หน่วยเวลา

1. Non-Preemptive SJF

Process	Arrival Time	Run Time	Start Time	Finish Time	Turnaround Time
P_1	0.0	8	0.0	8.0	8.0
P_2	0.4	4	9.0	13.0	12.6
P_3	1.0	1	8.0	9.0	8.0
					28.6

ค่าเฉลี่ย Turnaround Time = 28.6/3 = 9.53 หน่วยเวลา

2. Preemptive SJF (SRTF)

Process	Arrival Time	Run Time	Start Time	Finish Time	Left Time	Turnaround Time
P_1	0.0	8	0.0	0.4	7.6	0.4
P_2	0.4	4	0.4	1.0	3.4	0.6
P_3	1.0	1	1.0	2.0	0	1.0
P_2	1.0	3.4	2.0	5.4	0	4.4
P_1	0.4	7.6	5.4	13.0	0	12.6
						19.0

ค่าเฉลี่ย Turnaround Time = 19/3 = 6.33 หน่วยเวลา

การวิเคราะห์อัลกอริทึม SJF

- **ข้อดี**: ลดค่าเฉลี่ยของ Waiting Time และ Turnaround Time ได้อย่างมีประสิทธิภาพ โดยเฉพาะเมื่อมีโพรเซสที่ใช้เวลาสั้นจำนวนมาก
- **ข้อเสีย**: ต้องมีการพยากรณ์เวลาของ CPU Burst ล่วงหน้า ซึ่งในทางปฏิบัติไม่สามารถรู้ได้แน่ชัด
- **ปัญหา Starvation**: โพรเซสขนาดใหญ่ (Long Job) อาจต้องรอเป็นเวลานานมาก หากมีโพรเซสเล็ก ๆ เข้ามาอย่างต่อเนื่อง ทำให้ไม่ได้รับโอกาสใช้งานหน่วยประมวลผลกลาง ทันที
- **การใช้งานจริง**: มักถูกใช้ในระบบที่สามารถประมาณเวลาการประมวลผลได้คร่าว ๆ เช่น งานวิจัยเชิงวิทยาศาสตร์ หรือระบบที่จัดการงานเป็นชุด (Batch System)

6.4.3 Shortest-Remaining-Time-First (SRT)

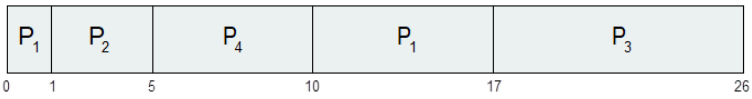
อัลกอริทึม **Shortest-Remaining-Time-First (SRT)** ถือเป็นการต่อยอดจากแนวคิดของ Shortest-Job-First (SJF) แต่ปรับมาใช้ในลักษณะ **Preemptive Scheduling** กล่าวคือ หากมีโพรเซสใหม่เข้ามาในระบบและใช้เวลาการประมวลผล (CPU Burst) ที่สั้นกว่าจำนวนเวลาที่เหลือของโพรเซสที่กำลังทำงานอยู่ ระบบจะ**หยุด (Preempt)** โพรเซสเดิมทันที และมอบสิทธิ์การใช้งานหน่วยประมวลผลกลางให้โพรเซสใหม่

ดังนั้น การตัดสินใจของ SRT ขึ้นอยู่กับ *เวลาที่เหลือ (remaining time)* ของโพรเซสที่กำลังทำงานเปรียบเทียบกับโพรเซสใหม่ที่เข้ามา

ตัวอย่างที่ 5 : สมมติว่าระบบมีโพรเซส 4 ตัว ดังนี้:

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

จากข้อมูลดังกล่าว หากใช้ SRT จะได้ Gantt Chart ดังแสดงในรูปนี้



ค่าเฉลี่ยของเวลารอคอย = $\dfrac{(10-1)+(1-1)+(17-2)+(5-3)}{4} = 6.5$ msec

ข้อสังเกต:

- SRT สามารถลดค่าเฉลี่ยของเวลารอ (Waiting Time) ได้ดีกว่า SJF แบบ Non-preemptive
- อย่างไรก็ตาม ปัญหาที่ตามมาคือ การอดตาย (Starvation) เนื่องจากโพรเซสที่มี Burst Time ยาวอาจถูกเลื่อนออกไปเรื่อย ๆ หากมีโพรเซสเล็ก ๆ เข้ามาอย่างต่อเนื่อง
- ในการใช้งานจริง อัลกอริทึมนี้มักใช้กับระบบที่ต้องการการตอบสนองเร็ว (Interactive System) แต่จำเป็นต้องมีกลไกเพิ่มเติม เช่น **Aging**เพื่อป้องกันการอดตายของโพรเซสขนาดใหญ่

6.4.4 Priority Scheduling

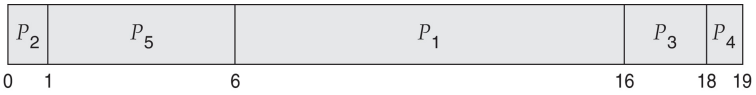
การจัดลำดับตามความสำคัญ (Priority Scheduling) เป็นอีกหนึ่งวิธีการจัดเวลาหน่วยประมวลผลกลาง โดยใช้*ระดับความสำคัญ (Priority)* เป็นเกณฑ์ในการเลือกโพรเซสที่จะได้รับสิทธิ์ใช้งานหน่วยประมวลผลกลางก่อน การกำหนดค่าลำดับความสำคัญอาจทำได้สองวิธี โดยขึ้นอยู่กับนโยบายของระบบปฏิบัติการ :

1. ตัวเลขน้อย = ความสำคัญสูง
2. ตัวเลขมาก = ความสำคัญสูง

ตัวอย่างที่ 6 : สมมติว่าโพรเซส 5 ตัวมี Burst Time และ Priority ดังนี้:

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

ผลการจัดลำดับสามารถแสดงด้วย Gantt Chart ดังรูปนี้



ค่าเฉลี่ยของเวลารอ = (6+0+16+18+1)/5 = 8.2 msec

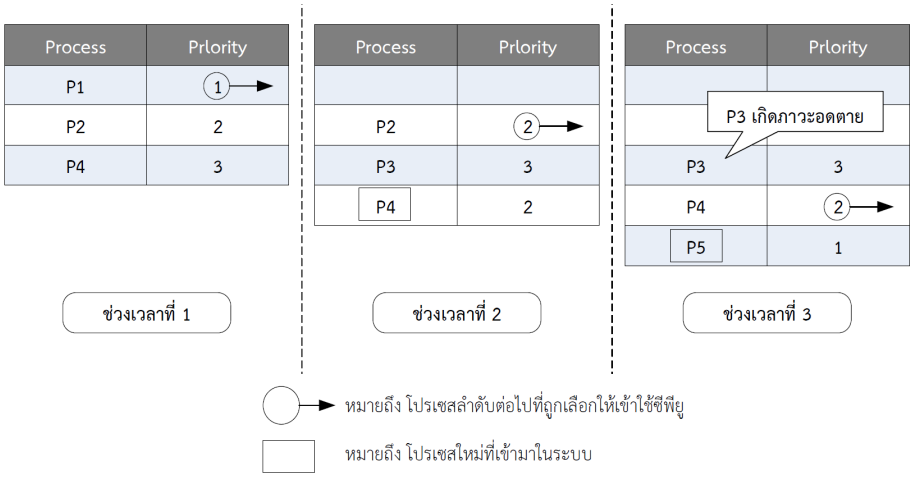
อัลกอริทึมนี้มีทั้งแบบ **Non-preemptive** และ **Preemptive** ในกรณีที่ใช้แบบ Preemptive จะเกิดปัญหาที่พบบ่อยคือ การอดตาย โดยโพรเซสที่มีลำดับความสำคัญต่ำกว่าอาจถูกเลื่อนออกไปเรื่อย ๆ และไม่ได้รับโอกาสใช้งานหน่วยประมวลผลกลางเลย

วิธีแก้ปัญหา: Aging เป็นเทคนิคที่เพิ่มค่าความสำคัญของโพรเซสที่รออยู่ในระบบทีละน้อยเมื่อเวลาผ่านไป ทำให้โพรเซสที่มีความสำคัญต่ำในตอนแรกมีโอกาสได้รับหน่วยประมวลผลกลางในที่สุด

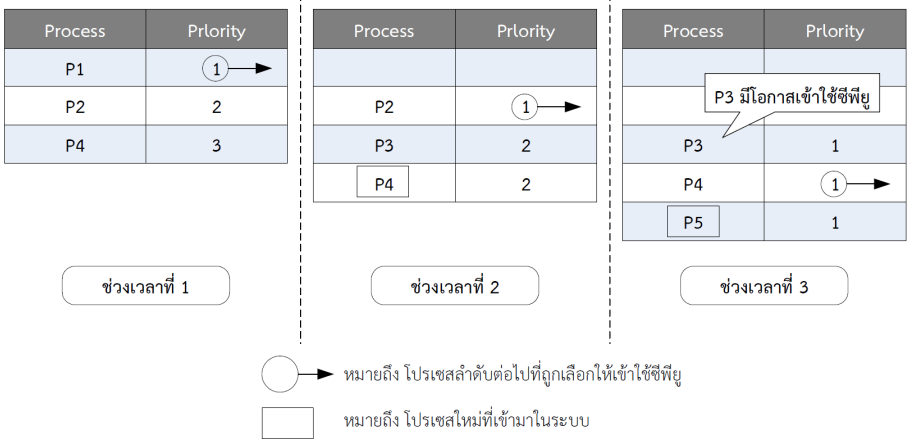
ตัวอย่างที่ 7 : พิจารณาโพรเซส P_1, P_2, P_3 ที่มี Priority ดังนี้ (โดยกำหนดให้ตัวเลขที่น้อยที่สุดมีความสำคัญสูงสุด):

Process	Priority
P_1	1
P_2	2
P_3	3

1. Priority Scheduling แบบ Preemptive โดย **ไม่มี Aging**

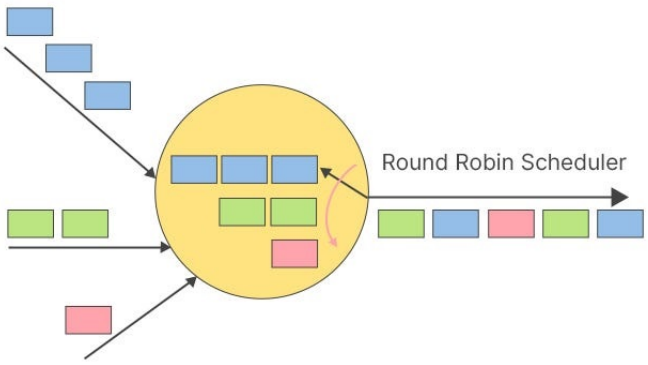


2. Priority Scheduling แบบ Preemptive โดย **มี Aging**



6.4.5 Round-Robin (RR)

อัลกอริทึม **Round-Robin (RR)** ถูกออกแบบมาสำหรับ **ระบบแบ่งเวลา (Time-sharing System)** โดยมีหลักการทำงานคล้าย FCFS แต่เพิ่มการกำหนดเวลาในการใช้งาน หน่วยประมวลผลกลาง ของแต่ละโพรเซส ซึ่งเรียกว่า **Quantum Time** หรือ **Time Slice**



รูปที่ 6.3: การทำงานของอัลกอริทึม Round-Robin

กลไกการทำงาน:

- Scheduler จะเลือกโพรเซสจากหัวคิวพร้อมเข้ามาใช้งานหน่วยประมวลผลกลาง ตาม Quantum ที่กำหนด
- หากโพรเซสเสร็จภายใน Quantum → คืนหน่วยประมวลผลกลางทันที
- หากไม่เสร็จ → ถูกหยุด (Preempt) และย้ายไปต่อท้ายคิว เพื่อรอทำงานรอบใหม่

ตัวอย่างที่ 8 : พิจารณาระบบที่มีโพรเซส 3 ตัว:

- P_1: เข้าระบบเวลา 0.0, Burst Time = 8
- P_2: เข้าระบบเวลา 0.4, Burst Time = 4
- P_3: เข้าระบบเวลา 1.0, Burst Time = 1

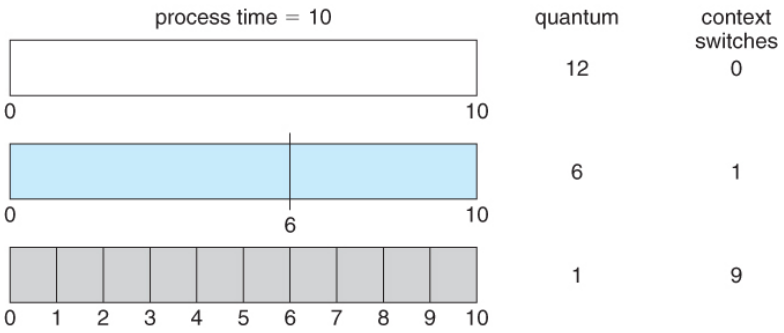
หาก Quantum = 2.0 หน่วยเวลา จะได้ตารางการทำงานดังนี้:

เวลา	โพรเซสที่ทำงาน
0.0–2.0	P_1
2.0–4.0	P_2
4.0–5.0	P_3 (เสร็จ)
5.0–7.0	P_1
7.0–9.0	P_2 (เสร็จ)
9.0–11.0	P_1
11.0–13.0	P_1 (เสร็จ)

เวลาครบวงจรเฉลี่ย (Average Turnaround Time) = 25.6/3 = 8.53 หน่วยเวลา

การวิเคราะห์อัลกอริทึม Round-Robin

- **ข้อดี:** เหมาะกับระบบ Interactive ผู้ใช้จะรู้สึกว่าโพรเซสของตนได้รับการประมวลผลตลอดเวลา
- **ข้อเสีย:** ถ้า Quantum เล็กเกินไป จะเกิด Context Switching บ่อย ทำให้ระบบเสียเวลามาก
- **ข้อสังเกต (ดังแสดงในรูปที่ 6.4):**
  - หาก Quantum มีค่ามาก → RR จะทำงานใกล้เคียงกับ FCFS
  - หาก Quantum มีค่าน้อยมาก → แต่ละโพรเซสรู้สึกเหมือนมีหน่วยประมวลผลกลางเป็นของตนเอง



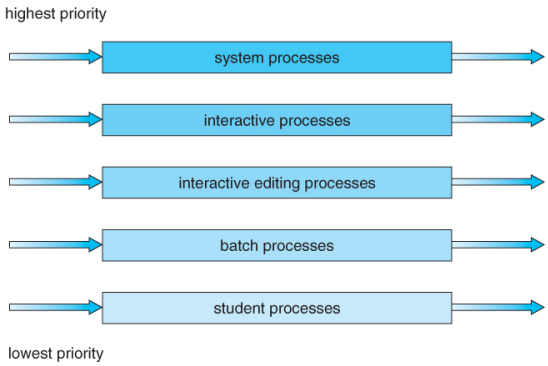
รูปที่ 6.4: ผลของขนาด Quantum ต่อ Context Switching

6.5 คิวหลายระดับ (Multilevel Queue Scheduling)

แนวคิดของการจัดตารางการทำงานแบบ **คิวหลายระดับ (Multilevel Queue)** เกิดจากข้อสังเกตว่าโพรเซสในระบบปฏิบัติการไม่ได้มีลักษณะเหมือนกันทั้งหมด แต่สามารถแบ่งออกเป็นหลายประเภทตามลักษณะการใช้งาน เช่น

- โพรเซสแบบทันที (Real-time processes) – ต้องการการตอบสนองที่แน่นอน
- โพรเซสของระบบ (System processes) – เกี่ยวข้องกับงานเบื้องหลังของระบบ
- โพรเซสแบบกลุ่ม (Batch processes) – งานที่ไม่เร่งด่วน มักทำงานเบื้องหลัง
- โพรเซสแบบโต้ตอบ (Interactive processes) – งานที่ผู้ใช้สั่งและคาดหวังผลลัพธ์เร็ว

โพรเซสแต่ละประเภทมีความต้องการด้าน**เวลาตอบสนอง (Response Time)** แตกต่างกัน ดังนั้นจึงไม่เหมาะที่จะใช้วิธีการจัดตารางแบบเดียวกันกับโพรเซสทุกประเภท แนวทางแก้ไขคือการแบ่งคิวพร้อม (Ready Queue) ออกเป็นหลายคิว โดยแต่ละคิวสงวนไว้สำหรับโพรเซสแต่ละกลุ่ม และมีอัลกอริทึมจัดเวลาเฉพาะของตนเอง ดังแสดงในรูปที่ 6.5



รูปที่ 6.5: การแบ่งโพรเซสตามระดับความสำคัญในการเข้าคิวหลายระดับ

จากรูปสามารถเห็นได้ว่ามีการแบ่งคิวออกเป็นหลายระดับ แต่ละคิวมีความสำคัญไม่เท่ากัน โพรเซสที่อยู่ในคิวความสำคัญสูงจะได้รับหน่วยประมวลผลกลางก่อนเสมอ ในขณะที่โพรเซสในคิวระดับต่ำจะถูกประมวลผลก็ต่อเมื่อโพรเซสในคิวที่สูงกว่าทำงานเสร็จทั้งหมดแล้ว

โดยทั่วไป การทำงานระหว่างคิวจะใช้**วิธีให้สิทธิ์สูงสุดก่อน (Fixed Priority Preemptive Scheduling)** เช่น คิวโพรเซสแบบโต้ตอบจะได้รับสิทธิก่อนคิวโพรเซสแบบกลุ่ม หากมีโพรเซสใหม่เข้ามาในคิวที่สำคัญกว่า ระบบจะหยุดการทำงานของโพรเซสที่อยู่ในคิวล่างลงทันที

อีกวิธีที่นิยมคือ **การแบ่งสัดส่วนเวลา (Time Slice among Queues)** เช่น กำหนดให้คิวโต้ตอบได้รับหน่วยประมวลผลกลาง 80% ของเวลา ส่วนคิวงานแบบกลุ่มได้รับ 20% วิธีนี้ช่วยให้โพรเซสทุกประเภทมีโอกาสทำงานบ้าง แม้จะอยู่ในคิวความสำคัญต่ำ

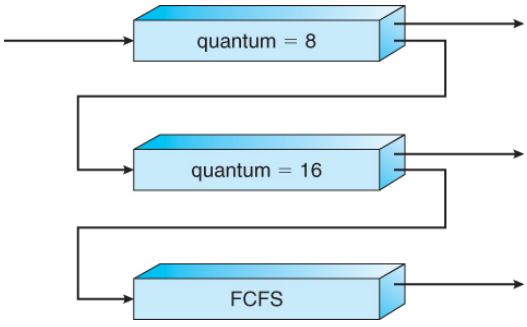
ตัวอย่างการแบ่งคิว 5 ระดับ:

- งานของระบบ (System processes)
- งานแบบโต้ตอบ (Interactive processes)
- งานแก้ไขข้อมูล (Interactive editing processes)
- งานแบบกลุ่ม (Batch processes)
- งานของนักศึกษา (Student processes)

การจัดเวลาทำงานแบบคิวหลายระดับที่เลื่อนชั้นได้ (Multilevel Feedback Queue)

ข้อจำกัดสำคัญของ Multilevel Queue คือ **โพรเซสไม่สามารถย้ายคิวได้** ซึ่งอาจไม่เหมาะสมในบางสถานการณ์ เพื่อแก้ปัญหานี้จึงมีการพัฒนา **Multilevel Feedback Queue (MLFQ)** ซึ่งโพรเซสสามารถเลื่อนขึ้นหรือลงระหว่างคิวได้ตามพฤติกรรมการใช้งานหน่วยประมวลผลกลาง (ดังแสดงในรูปที่ 6.6)

- ถ้าโพรเซสใช้หน่วยประมวลผลกลางนานเกินไป → ถูกเลื่อนลงไปคิวล่าง (เพื่อลดการรบกวนงานโต้ตอบ)
- ถ้าโพรเซสรอนานเกินไป → ถูกเลื่อนขึ้นคิวสูงกว่า (เพื่อลดโอกาสอดตาย: starvation)



รูปที่ 6.6: การจัดคิวหลายระดับแบบเลื่อนชั้น (MLFQ scheduling)

ตัวอย่างที่ 9 : หากระบบมี 3 คิว ได้แก่ แถวที่ 0, 1 และ 2

- คิวที่ 0: Quantum = 8 ms
- คิวที่ 1: Quantum = 16 ms

- คิวที่ 2: ใช้ FCFS สำหรับงานที่เหลือ

โพรเซสใหม่ทั้งหมดจะเข้าสู่คิวที่ 0 ก่อน หากทำงานเสร็จภายใน 8 ms จะถือว่าเป็นงานสำคัญและจบงานได้เร็ว หากไม่เสร็จ จะถูกเลื่อนลงไปต่อท้ายคิวที่ 1 และหากยังไม่เสร็จอีก จะถูกเลื่อนลงไปคิวที่ 2 วิธีนี้ทำให้โพรเซสที่ใช้หน่วยประมวลผลกลางมาก (CPU-bound) ค่อย ๆ ถูกเลื่อนลงไป ขณะที่โพรเซสแบบโต้ตอบ (I/O-bound) มีแนวโน้มทำงานเสร็จในคิวระดับสูง

พารามิเตอร์สำคัญของ MLFQ:

- จำนวนของคิวทั้งหมด
- อัลกอริทึมที่ใช้ในแต่ละคิว
- กฎการเลื่อนระดับขึ้น/ลง
- เงื่อนไขการเริ่มต้น (โพรเซสใหม่ควรเริ่มที่คิวใด)

MLFQ ถือเป็นอัลกอริทึมการจัดเวลาที่ทรงพลังและยืดหยุ่นมากที่สุด เพราะสามารถผสมผสานนโยบายหลายแบบเข้าด้วยกัน แต่ก็เป็นวิธีที่ซับซ้อนและต้องการการออกแบบพารามิเตอร์อย่างระมัดระวัง

6.6 การจัดตารางการทำงานสำหรับหลายหน่วยประมวลผล (Multi-processor Scheduling)

ในระบบที่มีหลายหน่วยประมวลผล การจัดตารางจะซับซ้อนมากขึ้น โดยแบ่งได้เป็น 2 กรณีหลัก:

- **Heterogeneous system** – หน่วยประมวลผลต่างชนิดกัน โพรเซสอาจทำงานได้เฉพาะบนหน่วยประมวลผลกลางบางตัว (เช่น Assembly ของ VAX รันบน IBM ไม่ได้) ดังนั้น ต้องแยกคิวสำหรับแต่ละประเภท
- **Homogeneous system** – ทุกหน่วยประมวลผลเหมือนกัน โดยสามารถใช้คิวร่วมกันได้

การใช้คิวร่วมกันช่วยให้โหลดงานกระจายตัวสมดุลมากขึ้น แต่ต้องแก้ปัญหาการใช้ข้อมูลร่วมกัน (Critical Section) โดยวิธีการมี 2 แนวทาง:

1. ให้แต่ละหน่วยประมวลผลกลาง ดึงงานจากคิวร่วมกัน (ต้องป้องกันการเลือกงานซ้ำ)
2. ใช้หน่วยประมวลผลกลางหนึ่งตัวทำหน้าที่เป็น **Master** คอยจัดตารางให้หน่วยประมวลผลกลางอื่น ๆ (เรียกว่า Master-slave model)

6.7 การจัดตารางแบบตอบสนองทันที (Real-time Scheduling)

การจัดตารางงานแบบเรียลไทม์แบ่งได้เป็น 2 ประเภทหลัก:

1. **Hard Real-Time** – ต้องเสร็จภายในเวลาที่กำหนดเสมอ หากทำไม่ได้ ระบบจะปฏิเสธงาน
2. **Soft Real-Time** – โพรเซสสำคัญจะได้สิทธิ์ก่อน แต่ไม่การันตีเวลาแน่นอน มักใช้ในงานมัลติมีเดียหรือกราฟิก

ปัญหาสำคัญในระบบเรียลไทม์คือ **Priority Inversion** ซึ่งเกิดเมื่อโพรเซสที่สำคัญสูงต้องรอให้โพรเซสความสำคัญต่ำปล่อยทรัพยากรก่อน วิธีแก้คือ **Priority Inheritance Protocol** ที่ให้โพรเซสที่มีความสำคัญต่ำชั่วคราวสืบทอด Priority สูงขึ้นจนจบงาน

6.8 การประเมินผลของอัลกอริทึม

การประเมินผลของอัลกอริทึม (Evaluation of Scheduling Algorithms) มีหลายวิธี เช่น:

- **Deterministic Modeling** – คำนวณจากชุดข้อมูลที่กำหนด (ง่ายแต่ใช้ได้เฉพาะกรณีตัวอย่าง)
- **Queuing Models** – วิเคราะห์ตาม Arrival rate และ Service rate ใช้สูตร Little ($n = \lambda W$)
- **Simulation** – เขียนโปรแกรมจำลองระบบ เพื่อศึกษาผลลัพธ์โดยใกล้เคียงความจริง
- **Implementation** – ทดลองจริงบนระบบ แต่มีค่าใช้จ่ายสูง

คำถามท้ายบท

1. ถ้าให้โพรเซสเข้าสู่ระบบตามตารางดังนี้

Process	Arrival Time	Burst Time (ms)
P_1	0	5
P_2	5	20
P_3	10	10
P_4	15	15

กำหนดให้โพรเซสเข้ามาตามลำดับคือ P_1, P_2, P_3 และ P_4 ให้นิสิตทำการแสดงการจัดเวลาหน่วยประมวลผลกลาง สำหรับโพรเซสตามอัลกอริทึม FCFS, SJF, Priority (ให้ตัวเลขโพรเซสที่มีค่าน้อยแสดงถึงค่า Priority สูง) และวิธีการ RR เมื่อ Quantum = 1 พร้อมทั้งหาค่าเฉลี่ยของเวลารอในคิวของแต่ละโพรเซส โดยใช้อัลกอริทึมแบบ FCFS, SJF และ RR